2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00084

BitMoD: Bit-serial Mixture-of-Datatype LLM Acceleration

Yuzong Chen[†], Ahmed F. AbouElhamayed[†], Xilai Dai[†], Yang Wang[‡], Marta Andronic[§], George A. Constantinides[§], and Mohamed S. Abdelfattah[†]

[†]Computer Systems Lab, Cornell University
[‡]Systems and Networking Research Group, Microsoft Research
[§]Department of Electrical and Electronic Engineering, Imperial College London

[†]{yc2367, afa55, xd44, mohamed}@cornell.edu
[‡]yang.wang92@microsoft.com
[§]{marta.andronic18, g.constantinides}@imperial.ac.uk

***Abstract*—Large language models (LLMs) have demonstrated remarkable performance across various machine learning tasks. Yet the substantial memory footprint of LLMs significantly hinders their deployment. In this paper, we improve the accessibility of LLMs through BitMoD[1], an algorithm-hardware co-design solution that enables efficient LLM acceleration at low weight precision. On the algorithm side, BitMoD introduces fine-grained data type adaptation that uses a different numerical data type to quantize a group of (e.g., 128) weights. Through the careful design of these new data types, BitMoD is able to quantize LLM weights to very low precision (e.g., 4 bits and 3 bits) while maintaining high accuracy. On the hardware side, BitMoD employs a bit-serial processing element to easily support multiple numerical precisions and data types; our hardware design includes two key innovations: First, it employs a unified representation to process different weight data types, thus reducing the hardware cost. Second, it adopts a bit-serial dequantization unit to rescale the per-group partial sum with minimal hardware overhead. Our evaluation on six representative LLMs demonstrates that BitMoD significantly outperforms state-of-the-art LLM quantization and acceleration methods. For discriminative tasks, BitMoD can quantize LLM weights to 4-bit with $< 0.5\%$ accuracy loss on average. For generative tasks, BitMoD is able to quantize LLM weights to 3-bit while achieving better perplexity than prior LLM quantization scheme. Combining the superior model performance with an efficient accelerator design, BitMoD achieves an average of $1.69\times$ and $1.48\times$ speedups compared to prior LLM accelerators ANT and OliVe, respectively.**

I. Introduction

Large language models (LLMs) have achieved significant breakthroughs in natural language processing tasks [47], [57]. However, the growth of LLM size and complexity continues to outpace the scaling of compute performance and memory capacity in existing hardware platforms [22]. For example, the first generation of the GPT model, introduced in 2018, contains only 117 million parameters, while the second and third generations grew more than $10\times$ and $1000\times$, respectively within two years [9]. This rapid increase in size necessitates significant memory capacity for model deployment, hindering their wide adoption, especially in edge scenarios with limited compute and memory resources. For instance, the state-of-the-art (SOTA) open-source LLM family, Llama-3 [35], contains more than 8 billion parameters and requires more than 16GB of memory to store the model weights in 16-bit floating-point (`FP16`) format, which cannot fit in an edge GPU such as Jetson-TX2 with 8GB memory [37]. Therefore, designing novel LLM compression algorithms, together with accelerators co-designed for efficient deployment of the compressed models, presents a promising solution to enhancing the accessibility of LLMs on edge devices.

Quantization serves as one of the most hardware-efficient methods to mitigate the computation and memory demands of LLMs. Generally, there are two types of quantization mechanisms. The first one is quantization-aware training (QAT), where retraining is needed to update model weights and quantization parameters (e.g., scaling factors) [26], [31]. The second approach is post-training quantization (PTQ), which does not require retraining [10], [19], [20], [25], [30], [42], [52]. Although QAT can achieve more competitive accuracy than PTQ, the prohibitive cost of retraining LLMs makes it less practical. As a result, PTQ is commonly adopted in existing LLM quantization studies. While some PTQ works quantize both weights and activations into low precision [25], [42], [52], weight-only quantization can offer a better trade-off between model accuracy and hardware efficiency for edge deployment of LLMs, where weights dominate the memory footprint [10], [19], [20], [30]. However, existing weight-only quantization works on GPUs suffer from poor computational efficiency since GPUs lack dedicated hardware to perform multiplication between integer weight and floating-point activation. Consequently, these methods must first dequantize the weight to FP16 and rely on the floating-point pipeline for computation.

To achieve better computational efficiency for LLMs, a recent accelerator work, FIGNA [27], proposes a family of dedicated computing units for mixed-precision arithmetic between integer weights and floating-point activations. To further unleash the potential of quantization for improved hardware efficiency, several works have proposed algorithm-hardware co-design solutions based on *custom* low-precision data types [25], [26], [38], [40]. The microscaling format (MX) [38], [40], assigns 8-bit metadata as the shared exponent to a group of low-precision weights. ANT [26] introduces a

[1]Code is available at: https://github.com/yc2367/BitMoD-HPCA-25

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00084

new data type that better adapts to the intra-tensor value distribution, thus reducing the quantization error. OliVe [25] proposes an outlier-victim-pair quantization mechanism, where an outlier value with a large magnitude is represented with an "Adaptive Biased Float" format and can be protected by pruning its adjacent victim value that has a small magnitude.

In this paper, we propose *BitMoD*[2], an algorithm-hardware co-design solution for efficient LLM acceleration at low weight precision. On the algorithm side, *BitMoD* exploits the *per-group* quantization [16], and modifies low-precision floating-point data types by repurposing the redundant zero value with a special value, which provides the ability to better adapt the data type itself to the numerical distribution of each weight group. Through careful choice of special values, *BitMoD* is able to quantize LLM weights to very low precision (e.g., 4-bit and 3-bit) with tiny encoding overhead while maintaining good model accuracy. On the hardware side, *BitMoD* employs the bit-serial computing paradigm with a unified representation for different low-precision data types to efficiently trade-off weight precision and hardware efficiency.

The main contributions of this paper are summarized below:

1) We propose *BitMoD*, a hardware-efficient PTQ solution for LLM acceleration. *BitMoD* introduces new data types that are tailored for per-group weight quantization at 4-bit and 3-bit precision with tiny encoding overhead.
2) We demonstrate that the proposed data types can be seamlessly integrated with other quantization optimization techniques, achieving better model perplexity than SOTA software-only LLM quantization works.
3) We propose an efficient accelerator design for *BitMoD*, which adopts a unified bit-serial representation for multiple low-precision data types. This effectively reduces the hardware cost to perform computation between low-precision weights and FP16 activations, and trades-off weight precision for improved hardware efficiency.
4) Our evaluation on six representative LLMs shows that on average, *BitMoD* achieves $2.2\times$ speedup and $2.31\times$ better energy efficiency compared to the baseline FP16 accelerator, *without* loss in accuracy. Compared to SOTA accelerators ANT and OliVe, *BitMoD* achieves an average speedup of $1.69\times$ and $1.48\times$, respectively.

II. Background and Motivation

A. Why Weight Quantization for LLMs?

To demonstrate the importance of LLM weight quantization for edge applications, we profile the total memory access footprint of weight and activation for four representative LLMs running both discriminative and generative tasks with a batch size of 1. For discriminative tasks, the LLM receives an input context and outputs a single token such as in sentiment analysis [46] and multiple-choice question answering [13]. For generative tasks, the LLM receive an input context and output multiple tokens. We set the input to output sequence length to 256 : 1 and 256 : 256 for discriminative and generative tasks, respectively, catering for edge applications as suggested by Lin *et al.* [30]. As shown in Fig. 1, the LLM weights access consumes orders of magnitude larger memory than the activations access. Although discriminative tasks only need to output a single token (e.g., "A"/"B"/"C" for multiple-choice question answering), the weight tensor dimension of an LLM (e.g., 2048 for OPT-1.3B) is much larger than the input token length, leading to memory access dominated by weights. Moreover, generative tasks necessitate repeated weight fetching for every new output token, resulting in significantly higher memory access for LLM weights. Thus, weight quantization is more effective for deploying LLMs in edge scenario where the batch size is small and the input token length is typically short.

[2]*BitMoD* stands for Bit-serial computation with Mixture of Data types.

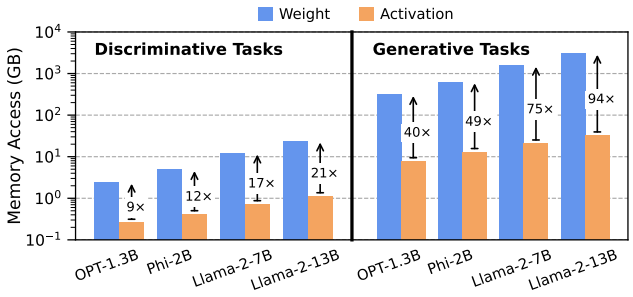

Fig. 1: Total memory access of weights and activations on discriminative tasks (with 256 input tokens and 1 output token) and generative tasks (with 256 input tokens and 256 generated tokens). Note the log scale on the y-axis. Note that the gap between weight and activation memory accesses increases for generative tasks at batch size 1 despite a much larger KV-cache than discriminative tasks. While prior work [44] has correctly reported a memory bottleneck caused by the KV-cache, this only occurs for 175B+ parameter models with a high batch size (e.g., 512) and a context lengths exceeding 512 tokens. This scenario is less relevant to our focus on low-batch edge LLM inference where the weights indeed dominate the total memory accesses.

B. Quantization Basics

One of the most popular quantization schemes is integer quantization, where a floating-point value is scaled and rounded to a low-precision integer. There are two widely used quantization modes – *symmetric* and *asymmetric*. Symmetric integer quantization can be expressed as follows:

$$\Delta = \frac{W_{f\max}}{2^{b-1}-1};\ W_q = \texttt{Round}\left(\frac{W_f}{\Delta}\right);\ W_{qf} = W_q \cdot \Delta \quad (1)$$

where W_f is the original floating-point tensor, $W_{f\max}$ is the absolute maximum value, b is the quantized integer precision, Δ is the scaling factor, W_q is the quantized integer value, and W_{qf} is the floating-point value after performing dequantization (i.e., re-scaling).

The symmetric quantization assumes that the minimum and maximum values of a tensor have the same absolute value (i.e., symmetric value range), but this is not always true. Hence, another popular mode of quantization is asymmetric quantization, which can be expressed as follows:

$$\Delta = \frac{\texttt{Range}\,(W_f)}{2^b-1};\quad z = \texttt{Round}\left(\frac{-W_{f\min}}{\Delta}\right)$$
$$W_q = \texttt{Round}\left(\frac{W_f}{\Delta}\right) + z;\quad W_{qf} = (W_q - z)\cdot\Delta \quad (2)$$

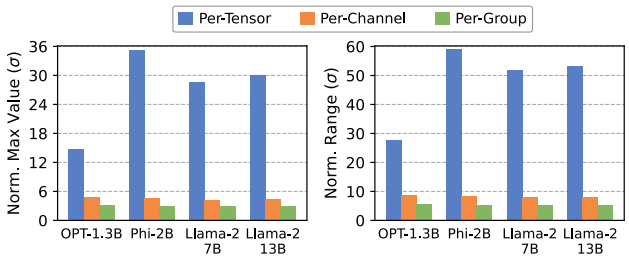

Fig. 2: Maximum value and value range for different quantization granularity. Results are normalized to the standard deviation (σ) of the weight vector at the corresponding granularity, then averaged across all weight vectors. The per-group granularity has a group size of 128.

where $W_{f\text{min}}$ is the absolute minimum value of W_f, and z represents the zero-point of the quantized tensor.

C. Motivation

We analyze several techniques that are widely adopted in recent quantization studies, which motivates our proposed *BitMoD* framework. We mainly focus on weight quantization in our discussion.

Quantization Granularity Matters. Consider a floating-point weight tensor $W_f^{K\times D}$, where K represents the number of output channels and D is the channel size. There are three *granularities* to quantize the model weight: per-tensor, per-channel, and per-group. The *per-tensor* quantization uses the same scaling factor to quantize a whole weight tensor, while *per-channel* quantization divides the weight tensor along the output channel into K vectors, and quantizes every vector $W_f^{1\times D}$ independently. However, given the large tensor size and hidden dimension of LLMs, these two granularities still lead to large quantization error. Specifically, the quantization error of a dequantized weight in Eq. 1 can be expressed as:

$$\texttt{Error}(W_{qf}) = \texttt{ErrorRound}\left(\frac{W_f}{\Delta}\right)\cdot\Delta \qquad (3)$$

where ErrorRound is the rounding error during quantization, which has been shown to have an expected value of 0.25 [30]. Therefore, the quantization error is proportional to the scaling factor Δ, which is further proportional to the maximum value and range for symmetric (Eq. 1) and asymmetric (Eq. 2) quantization, respectively.

In order to further reduce the quantization error, recent LLM quantization studies adopt the *per-group* granularity [16], [19], [20], [30]. The per-group quantization further divides a weight channel $W_f^{1\times D}$ into D/G groups, each with a group size of G. The group size introduces extra overhead to store the quantization parameters, i.e., scaling factor (and zero-point) for every group, and is usually set to 128 in SOTA quantization frameworks to balance accuracy and memory overhead [20], [30]. Fig. 2 demonstrates the benefits of per-group quantization by showing the maximum value and range in four representative LLMs at different granularity. The per-group granularity has the lowest maximum value and range, hence will have a lower quantization error compared to the other two granularity. Therefore, we focus on per-group quantization in this work.

TABLE I. Wikitext-2 perplexity (↓) under different quantization granularity and 4-bit data types. "PC" and "PG" stand for per-channel and per-group, respectively. The group size is 128.

Model	OPT-1.3B		Phi-2B		Llama-2-7B		Llama-2-13B	
Granularity	PC	PG	PC	PG	PC	PG	PC	PG
FP16	14.62	14.62	9.71	9.71	5.47	5.47	4.88	4.88
INT4-Sym	36.05	16.04	13.03	11.15	12.92	5.84	5.47	5.07
INT4-Asym	48.41	15.41	12.08	**10.67**	8.89	**5.77**	5.27	**5.01**
FP4	16.07	**14.99**	**11.24**	10.68	8.07	**5.77**	**5.15**	5.05
Flint	**15.87**	16.23	11.71	11.23	**6.67**	6.09	5.31	5.29

Quantization Data Type Matters. Numerous studies have proposed custom data types for quantization at the per-channel granularity [25], [26], [40]. We analyze the effects of adopting different data types for per-channel and per-group weight quantization. We explore four basic data types at 4-bit precision: integer with symmetric (INT4-Sym) and asymmetric (INT4-Asym) quantization, floating-point (FP4), and the Flint data type proposed by ANT [26]. Table I shows the resulting perplexity on the Wikitext-2 dataset [33]. We highlight two important observations. First, although Flint can achieve better perplexity at the per-channel granularity, it never outperforms other data types at the per-group granularity. Second, the per-group INT4-Asym and FP4 quantization achieve the best perplexity on some but not all studied LLMs, indicating that both asymmetry and FP data types are favorable for per-group quantization. The reason behind this is twofold. First, weight tensors typically exhibit Gaussian-like distribution that fits well to the floating-point data type [17], [51]. Second, while the effects of outliers are mitigated by per-group quantization, a weight group can still contain outliers in an asymmetric pattern, being either solely positive or negative, as highlighted in previous studies [15], [16]. This characteristic benefits from asymmetric quantization.

The above observation motivates us to explore new quantization data types that can combine the benefits of *asymmetry* and FP formats to achieve better accuracy under per-group quantization. We notice that the basic FP data types have symmetric quantization values due to the inherent sign-magnitude binary representation that contains positive and negative zero values. Our key insight is that we can introduce additional asymmetry to FP by repurposing a redundant zero value with another *special value*. This approach provides us with two key benefits. First, it allows to fully utilize the limited quantization levels. Although the redundant zero value does not affect high-precision formats such as FP16, it constitutes a large fraction of quantization levels at low precision (e.g., 12.5% at 3-bit precision). Second, we can tune the special value to make the extended FP data types better adapt to the per-group weight distribution, which we discuss in Section III-B.

Quantization Bit-width Matters. While prior LLM accelerators mainly rely on bit-parallel architectures that support 8-bit and 4-bit precision [25]–[27], recent studies have shown that 6-bit floating-point weights exhibit negligible accuracy loss across various LLM models and tasks [49], [51]. Motivated by this, we analyze the effects of using different 6-bit data types for per-group LLM weight quantization. We consider four data

TABLE II. Wikitext-2 and C4 perplexity (↓) under different 6-bit data types. We use per-group weight quantization with a group size of 128.

Model	OPT-1.3B		Phi-2B		Llama-2-7B		Llama-2-13B	
Dataset	Wiki	C4	Wiki	C4	Wiki	C4	Wiki	C4
FP16	14.62	14.72	9.71	12.74	5.47	6.97	4.88	6.47
INT6-Sym	**14.51**	14.80	9.85	12.82	**5.49**	**6.99**	**4.89**	**6.46**
INT6-Asym	14.61	14.78	**9.76**	**12.8**	**5.49**	**6.99**	**4.89**	**6.46**
FP6-E2M3	14.59	**14.76**	9.85	**12.8**	5.52	**6.99**	4.92	6.49
FP6-E3M2	14.81	14.81	9.81	12.87	**5.49**	7.02	**4.89**	6.50

types: integer with symmetric (INT6-Sym) and asymmetric (INT6-Asym) quantization, floating-point with 2-bit exponent and 3-bit mantissa (FP6-E2M3), and floating-point with 3-bit exponent and 2-bit mantissa (FP6-E3M2). Table II compares the resulting perplexity of different quantization data types on Wikitext-2 [33] and C4 [18] datasets. On average, the studied 6-bit data types achieve similar and negligible perplexity loss compared to the FP16 baseline. For example, the average perplexity loss of INT6-Sym is less than 0.05, and its simple integer representation offers a promising solution to efficient LLM acceleration. Therefore, it is crucial for an accelerator to support diverse quantization bit-width to offer a better trade-off between memory footprint and model accuracy.

A natural solution for accommodating variable precision is to adopt bit-serial architectures [3], [12], [28], [45]. However, existing bit-serial accelerators mainly target the integer data type, which causes significant accuracy loss at 3-bit precision as we will show in Section V-B. Furthermore, these accelerators cannot leverage per-group quantization for improved accuracy. This is because per-group quantization assigns different scaling factors for different groups, necessitating a floating-point unit with large area overhead to dynamically dequantize the partial sum after computing the dot-product for every group. Thus, an efficient dequantization mechanism with low hardware cost is desirable.

Algorithm-Hardware Co-Design Matters. Numerous frameworks have been proposed to accelerate LLM execution, as depicted in Table III. SOTA algorithmic solutions such as AWQ [30] quantize LLM weights to low-precision integer while preserving high accuracy. Nevertheless, AWQ is optimized for LLM acceleration on GPUs, which lack dedicated mixed-precision computing unit. As a result, it converts the low-precision weights to FP16 and relies on the GPU floating-point pipeline for computation, resulting in poor computational efficiency.

In contrast, ANT [26], OliVe [25], and FIGNA [27] propose efficient bit-parallel accelerators for quantized model acceleration. But their precision is limited to 8-bit and 4-bit, which restricts the ability to utilize other precision (e.g., 6-bit) for a better accuracy-efficiency trade-off. Moreover, their accelerators do not natively support per-group quantization, which requires a floating-point unit to dynamically dequantize the per-group partial sum on the fly. While Microscaling [40] accommodates diverse precision, it necessities a floating-point pipeline to handle the shared micro-exponent of a weight group, leading to higher energy consumption compared to other low-precision compute units. Furthermore, given the significant memory footprint of LLMs, it is desirable to explore sub-4-bit quantization while maintaining good model accuracy, which ANT, OliVe, and Microscaling do not address. As we will show in Section V-B, the custom quantization data types proposed by ANT, OliVe, and Microscaling fail to achieve better accuracy than the simple asymmetric integer quantization at 4-bit weight precision, and cause unacceptable accuracy loss at 3-bit weight precision under per-group quantization. The above limitation motivates us to propose an efficient LLM acceleration framework that supports a wide range of hardware-friendly bit-widths, while maintaining good accuracy at low weight precision.

TABLE III. Comparison between *BitMoD* and SOTA co-design frameworks for LLM acceleration

Framework	Per-group Quant?	Supported Precision	Accuracy @ 3-bit Weight	Hardware Efficiency
AWQ [30]	Yes	Limited	High	Low
FIGNA [27]	No	Limited	Low	High
ANT [26]	No	Limited	Low	High
OliVe [25]	No	Limited	Medium	High
Microscaling [40]	Yes	Many	Low	Medium
***BitMoD* (Ours)**	Yes	Many	High	High

III. BitMoD Quantization Framework

In this section, we present the *BitMoD* quantization framework, which includes new data type families tailored for per-group quantization at 3-bit and 4-bit precision. Section III-A describes our proposed data types that extend the basic floating-point data types at 3-bit and 4-bit precision. Section III-B presents an enhanced per-group LLM quantization strategy using the proposed data types. Section III-C describes the hardware-efficient per-group dequantization mechanism using integer scaling factors.

A. Asymmetric FP3 and FP4 Data Types

The basic floating-point formats contain a redundant quantization level due to the sign-magnitude representation that has both $+0$ and -0. We propose to replace this redundant zero with another *special value* to fully utilize the available quantization levels and introduce additional asymmetry. We first use the basic FP3 format to derive our custom 3-bit data type, and then extend our idea to 4-bit precision.

FP3 Extension. The basic FP3 data type contains seven distinct values $\{0, \pm1, \pm2, \pm4\}$. Our main idea is to extend FP3 and allows the redundant zero to be replaced by one of some pre-defined special values. Consequently, a weight group can be quantized by the basic FP3 data type together with a selected special value to minimize the quantization error. Ideally, the special values can have an arbitrary precision. But a high-precision (e.g., FP16) special value leads to more hardware overhead for computing, which offsets the efficiency of low-precision data types. Hence, we limit the special value to low-precision integers. Furthermore, given N as the number of allowed special values, an encoding overhead of $\lceil \log N \rceil$

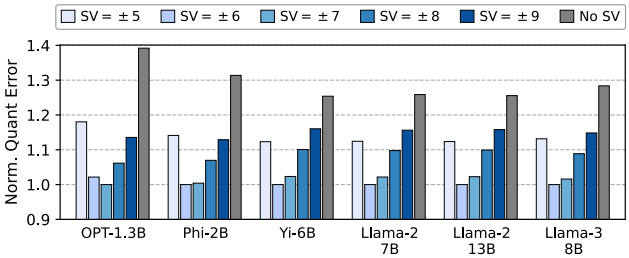

Fig. 3: Normalized weight quantization error (↓) with different special values (SV) for FP3. We use per-group quantization with a group size of 128. The special values ± 6 achieve the lowest overall quantization error, thus adopted in *BitMoD*.

bits is needed to specify which special value to be selected during computation. This selection also requires an N-to-1 mux in the hardware implementation. To balance the encoding overhead and hardware complexity, we set $N = 4$ which only requires 2-bit encoding per group.

The choice of special values will affect the resulting quantization error because it changes the set of available quantization values. As discussed in Section II-C, both asymmetry and floating-point data types are crucial for good accuracy under per-group quantization. Since the scaling factor and quantized values are ultimately determined by the absolute maximum value of a data type [32], we establish the set of special values based on two principles. First, some special values should fall inside the numerical range of FP3 to ensure that they do not alter its original absolute maximum (i.e., 4). This is advantageous for quantizing weight groups exhibiting symmetric, Gaussian-like distribution. Second, some special values could fall outside the numerical range of FP3 to introduce additional asymmetry, i.e., the absolute maximum and minimum quantization values of the extended FP3 are different. This can benefit weight groups that exhibit asymmetric distribution.

To satisfy the first property, the special values should be set to ± 3, which replace the redundant zero with $+3$ and -3, respectively. We call this new data type FP3-ER since it adds extra resolution (ER) within the range of FP3. To satisfy the second property, there are an infinite number of values that can fall outside the FP3 range. Therefore, we determine the two remaining special values that can minimize the quantization error. We further reduce the search space by restricting these two special values to have the same absolute value, which results in *balanced* asymmetry across all weight groups. This is desirable because, although an individual weight group may prefer asymmetric quantization, a whole weight tensor

TABLE IV. Our proposed extended resolution (ER) and extended asymmetry (EA) FP3 and FP4 data types.

Dtype	Basic Values	Extended Dtype	Special Value
FP3	$0, \pm 1, \pm 2, \pm 4$	FP3-ER	-3 or $+3$
		FP3-EA	-6 or $+6$
FP4	$0, \pm 0.5, \pm 1, \pm 1.5$ $\pm 2, \pm 3, \pm 4, \pm 6$	FP4-ER	-5 or $+5$
		FP4-EA	-8 or $+8$

Algorithm 1: Fine-grained data type adaptation.

Input : Weight group: W; Quantization precision: p
Output : Quantized weight group: W_{qout};
Selected special value: v_{out}

1 **Func** AdaptiveQuant(W, p) **:**
// Get basic and special quantization values according to Table IV
2 basicValues = GetBasicValues(p)
3 specialValues = GetSpecialValues(p)
// Search for the best special value
4 minError = $+\infty$
5 **for** v in specialValues **do**
6 quantValues = basicValues $\cup$ v
7 W_q = NonLinearQuantize(W, quantValues)
8 newError = MeanSquareError(W, W_q)
9 **if** newError $<$ minError **then**
10 minError = newError
11 $W_{qout} = W_q$
12 $v_{out} = v$
13 **return** W_{qout} , v_{out}

usually exhibits symmetric, Gaussian-like distribution [26], [55]. Fig. 3 shows the normalized per-group quantization error on six LLMs when adding different special values to FP3. We observe that adding asymmetry significantly reduces the quantization error. In addition, the special values ± 6 have the lowest quantization error on most LLMs except for OPT-1.3B, and are therefore adopted in *BitMoD*. We call the resulting new data type FP3-EA since it adds extra asymmetry (EA) to extend the range of FP3.

FP4 Extension. Similar to FP3-ER and FP3-EA, we add extra resolution and asymmetry to FP4. We conduct experiments to measure the effects of different FP4 special values on the resulting quantization error, which leads to the best FP4-ER and FP4-EA that have special values ± 5 and ± 8, respectively. Table IV summarizes the extended FP3 and FP4 data types. Note that although we have fixed the four special values given that they can minimize the quantization error for the diverse set of LLMs that we evaluate, the proposed *BitMoD* accelerator can flexibly accommodate other arbitrary special values that may perform well with different LLMs, which we discuss in Section IV-A.

B. Fine-grained Data Type Adaptation

The extended FP3 and FP4 data types contain four special values, but every weight group can only be quantized with one special value in addition to the basic values. Therefore, we propose a *fine-grained data type adaptation* strategy that quantizes every group using a different special value to minimize the quantization error, as detailed in Algo. 1. First, the basic and special values are obtained from Table IV (Line 2 – 3). We iterate through all special values and add one special value to the set of basic values in every iteration (Line 5 – 6). Then we perform non-linear quantization (Line 7), which is commonly used in previous PTQ studies that map the floating-

TABLE V. Wikitext-2 and C4 perplexity (↓) under different precision for the per-group scaling factor (SF). We use `INT4-Asym` for weight quantization with a group size of 128.

Model	OPT-1.3B		Phi-2B		Llama-2-7B		Llama-2-13B	
SF Bits	Wiki	C4	Wiki	C4	Wiki	C4	Wiki	C4
FP16	15.41	15.84	10.68	13.66	5.77	7.31	5.01	6.62
INT8	15.41	15.84	10.68	13.66	5.77	7.31	5.01	6.62
INT6	15.43	15.84	10.74	13.71	5.77	7.31	5.01	6.62
INT4	15.52	15.93	10.76	13.73	5.77	7.36	5.03	6.64
INT2	18.46	18.61	15.68	18.32	8.41	10.63	6.19	8.12

point tensor to a set of non-linear values (i.e., non-INT data types) [19], [25], [26]. Finally, we assign the special value that has the lowest mean-square error between the original and quantized weight (Line 8 – 11).

Although Algo. 1 describes the quantization procedure for a single weight group, the algorithm can be vectorized on a GPU to find the best special value for all groups of a weight tensor simultaneously. For our implementation, the algorithm only takes ∼10 second to quantize the whole Llama-2-7B model on a single A6000 GPU. Hence, our proposed quantization strategy exhibits high compression speed and efficiency.

C. Efficient Per-group Dequantization

While quantization allows computing the dot product in low precision, it still requires dequantization (i.e., re-scaling) after producing the output. Specifically, Eq. 1 indicates that the quantized low-precision weight should be multiplied by the scaling factor to obtain the actual floating-point weight. Per-channel quantization only needs re-scaling after producing the final output activation. Such per-channel re-scaling can be further fused into other element-wise operations such as layer-norm before writing the output activation back to memory, reducing the data transfer cost [30], [52]. However, per-group quantization must dequantize the partial sum after computing every group of dot-products because different groups have different scaling factors. Furthermore, since *BitMoD* maintains the input activation at `FP16`, the group partial sum will also have floating-point formats. As a result, performing dequantization on-the-fly necessitates a floating-point pipeline, which can diminish the potential hardware efficiency gained from using low-precision weights.

In order to reduce the dequantization cost, we build upon prior work VS-Quant [14], which applies a second-level quantization that further quantizes the scaling factors to low-precision integers. Given the weight channel size D and group size G, VS-Quant applies symmetric quantization (Eq. 1) to the D/G scaling factors from the same channel, where the precision of per-group scaling factor is a design parameter. However, VS-Quant only targets small-scale neural networks and uses a small group size of 16. It is unclear how quantizing the scaling factors of a larger group will affect the accuracy of LLMs. Hence, we conduct experiments to find the best precision for per-group scaling factors. We use `INT4-Asym` quantization as an example, while other data types show the same trend. As shown in Table V, `INT8` per-group scaling factors have no accuracy loss compared to using `FP16` scaling

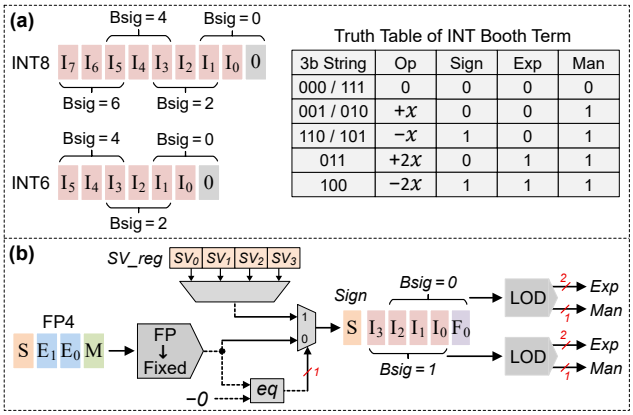

Fig. 4: Unified bit-serial representation of (a) INT8, INT6, and (b) FP4. Every bit-serial term contains four parts: sign, exponent (exp), mantissa (man) and bit-significance (bsig).

factors. This is expected since `INT8` can even achieve no accuracy loss for per-channel weight quantization [52], [53], which has a much wider numerical range than scaling factors. Thus, we use `INT8` per-group scaling factors in *BitMoD*, which allows efficient per-group dequantization in a bit-serial manner as will be described in Section IV-B.

Memory Overhead Analysis. The proposed *BitMoD* quantization only needs an 8-bit scaling factor and 2-bit encoding metadata to select the special value for every group. Given a large group size such as 128, which is commonly used in SOTA software-only LLM quantization studies [19], [20], [30], the 10-bit extra memory per group incurs practically no overhead. Furthermore, prior software-only PTQ works mainly adopt asymmetric integer quantization [19], [20], [30], which requires a 16-bit scaling factor and an 8-bit zero-point per group. Hence, *BitMoD* exhibits lower memory overhead compared to these works.

IV. BitMoD Hardware Accelerator

In this section, we describe the *BitMoD* hardware design, which leverages the bit-serial computing paradigm to offer a good trade-off between weight precision, model accuracy and hardware efficiency. Section IV-A develops a unified bit-serial representation of different low-precision data types supported by *BitMoD*. Section IV-B details the microarchitecture of the *BitMoD* processing element (PE). Section IV-C presents the overall accelerator architecture.

A. Unified Bit-serial Representation

Prior studies have demonstrated that per-channel `INT8` weight quantization shows no accuracy loss compared to using `FP16` weights [27], [52], [53]. Moreover, as discussed in Section II-C, per-group `INT6` quantization also has negligible accuracy loss. Hence, the design target of *BitMoD* hardware is to support `INT8`, `INT6`, as well as the new `FP4` and `FP3` extensions in a unified architecture. However, the basic values of `FP3` and `FP4` use the floating-point format, which is not compatible with the integer representation. A naive approach

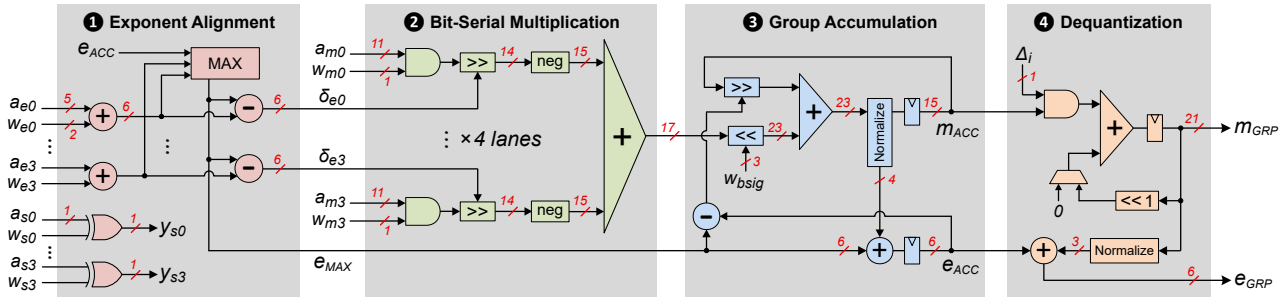

Fig. 5: The microarchitecture of *BitMoD* PE. Every bit-serial weight term contains 1-bit sign (w_s), 2-bit exponent (w_e), 1-bit mantissa (w_m), and a shared bit-significance (w_{bsig}).

is to convert all data types to `INT8`, yet this cannot improve the computational efficiency for lower-precision weights.

In order to trade-off lower weight precision for improved hardware efficiency, we propose a unified bit-serial representation, where every number is decomposed into a series of bit-serial terms, each containing four parts: sign, exponent (exp), mantissa (man), and bit-significance (bsig). The value of a bit-serial term can be expressed as:

$$v_{\text{term}} = (-1)^{\text{sign}} \cdot 2^{\text{exp}} \cdot \text{man} \cdot 2^{\text{bsig}} \tag{4}$$

Fig. 4 describes the bit-serial representation for different data types supported by *BitMoD*. For `INT8` and `INT6`, we apply Booth encoding [8] to decompose their binary strings into four and three 3-bit Booth strings, respectively. The Booth encoding has been widely used in prior bit-serial accelerators to speed up computation [3], [5], [43]. Every two adjacent Booth strings have a difference of 2 in bit-significance. The sign, exponent, and mantissa depend on the content of a Booth string, which defines the desired operation when multiplying with another operand x.

For the extended `FP4`, we first convert it to fixed-point values in sign-magnitude format with 1 sign bit, 4 integer bits $\{I_3, I_2, I_1, I_0\}$ to handle the largest special value ± 8 of `FP4-EA`, and 1 fraction bit $\{F_0\}$ to handle ± 0.5 and ± 1.5 of the basic `FP4` values. The fixed-point value is then compared with the redundant negative zero. If the comparison result is equal, the negative zero will be replaced by the assigned special value (*SV*) of a particular weight group. The four allowed special values are stored in a register file (*SV_reg*), which only requires one-time programming before deploying an LLM. To obtain the bit-serial term, we observe that all values of the extended `FP4` in Table IV contain at most two '1' bits after converting to the fixed-point format. Hence, we use the simple leading-one detector (*LOD*) to get two bit-serial terms from the first four bits $\{I_3, I_2, I_1, I_0\}$ and last four bits $\{I_2, I_1, I_0, F_0\}$, respectively. Finally, since the extended `FP3` values are a subset of `FP4`, it can be decoded into two bit-serial terms using the same hardware. Note that the bit-serial decoder is not limited to support the special values shown in Table IV. The special value register file can be programmed with other special values as needed, and the number of decoded bit-serial terms can be minimized with simple modification to the decoder. For example, the special value 7 can be expressed as two bit-serial terms 2^3 and -2^0 instead of using a leading-one detector that emits three bit-serial terms.

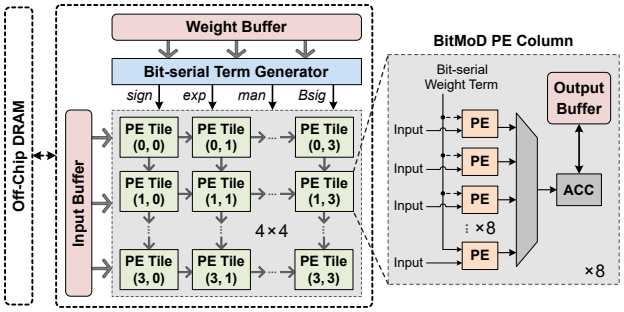

Fig. 6: *BitMoD* accelerator architecture.

B. BitMoD Processing Element

While *BitMoD* is able to quantize weight to low precision, activation still remains in `FP16`. To address this challenge, we propose a mixed-precision bit-serial PE as shown in Fig. 5. In every cycle, the PE performs a 4-way dot product between four bit-serial weight terms (w) and four `FP16` activations (a). Step ❶ first aligns the sum of exponents ($a_e + w_e$) to compute the delta exponent (δ_e). It also generates the sign (y_s) of every product between a weight term and activation. Step ❷ performs the bit-serial multiplication between the 1-bit weight mantissa (w_m) and 11-bit activation mantissa (a_m) including the hidden bit. The multiplication result is aligned by a right-shifter that is controlled by the delta exponent. We reserve 3 extra bits in the shifter result to account for rounding to the nearest even as suggested by Awad *et al.* [5]. The bit-serial dot product of the mantissa is then computed using an adder tree. After producing the bit-serial dot product, Step ❸ performs accumulation by first multiplying the dot product with the weight bit-significance (W_{bsig}), followed by adding with the accumulator mantissa (m_{ACC}). The accumulated mantissa is then normalized to update the accumulator exponent (e_{ACC}). Since *BitMoD* adopts per-group quantization, the accumulated group partial sum must be dequantized on the fly. To reduce this hardware cost, Step ❹ performs dequantization in a bit-serial manner. Specifically, the accumulator mantissa is multiplied by one bit of the group scaling factor (Δ_i) in every cycle, followed by shift-and-add to obtain the exponent (e_{GRP}) and mantissa (e_{GRP}) of the dequantized partial sum.

TABLE VI. Wikitext-2 and C4 perplexity (↓) using different data types under per-group weight quantization.

Precision	Datatype*	OPT-1.3B		Phi-2B		Yi-6B		Llama-2-7B		Llama-2-13B		Llama-3-8B		Mean ΔPPL
		Wiki	C4	Wiki	C4	Wiki	C4	Wiki	C4	Wiki	C4	Wiki	C4	
16-bit	FP16	14.62	14.72	9.71	12.74	5.84	8.91	5.47	6.97	4.88	6.47	6.13	8.88	0
4-bit	ANT	16.23	16.08	11.23	14.31	6.87	10.95	6.09	7.71	5.31	6.91	7.58	11.04	1.23
	OliVe	15.38	15.82	10.49	**13.51**	6.55	9.84	5.91	7.34	5.13	6.74	6.89	9.91	0.68
	MX-FP4	15.39	15.81	10.72	13.72	6.62	10.24	5.82	7.39	5.11	6.71	7.04	10.13	0.79
	INT4-Asym	15.41	15.74	10.67	13.65	6.32	9.69	5.77	7.31	**5.01**	6.62	6.84	9.79	0.62
	BitMoD	**14.89**	**15.29**	**10.48**	13.53	**6.23**	**9.58**	**5.72**	**7.26**	**5.01**	**6.61**	**6.73**	**9.66**	**0.48**
3-bit	ANT	340.6	332.9	15.57	18.35	9.01	14.32	8.51	10.28	6.40	7.98	15.22	17.56	57.61
	OliVe	76.79	59.63	14.93	17.76	32.42	66.02	9.13	12.04	8.69	12.43	26.76	46.39	23.14
	MX-FP3	1E+3	771.6	17.89	20.37	15.41	21.97	8.86	11.99	7.19	9.13	23.82	31.39	152.8
	INT3-Asym	139.4	144.9	13.92	16.79	8.66	13.33	7.08	9.29	5.64	7.35	13.26	17.80	24.34
	BitMoD	**22.67**	**20.47**	**12.91**	**15.69**	**7.66**	**11.98**	**6.55**	**8.36**	**5.50**	**7.18**	**8.96**	**12.82**	**2.94**

* All quantization data types use per-group quantization. The MX data type uses a group size of 32 following the standard in [38], while other data types use a group size of 128. The perplexity of MX degrades when using a larger group size.

One concern of the bit-serial dequantization is whether it will take more cycles than the normal dot product stage and cause potential pipeline stalling. As discussed in Section III-C, the per-group scaling factor has 8 bits, which requires 8 cycles for dequantization. On the other hand, even the lowest-precision data type `FP3` in *BitMoD* requires two cycles to process two bit-serial terms. Given the PE dot-product size of 4 and a commonly used group size of 128, the group dot-product stage takes $128/4 \times 2 = 64$ cycles to complete. Therefore, the proposed bit-serial dequantization will never stall the computing pipeline. Furthermore, since `INT6` and the extended `FP4`/`FP3` data types contain three and two bit-serial terms, the proposed *BitMoD* PE is able to compute 4 multiply-accumulate operations in 3 and 2 cycles, respectively. Compared to the normal `FP16` multiply-accumulate hardware, *BitMoD* achieves a throughput improvement of $1.33\times$ and $2\times$ for `INT6` and `FP4`/`FP3` data types, respectively. In fact, as will be evaluated in Section V-C, the *BitMoD* PE consumes 24% less area than an `FP16` PE, thus is able to provide even higher throughput under an iso-compute area constraint.

Besides matrix multiplication between weights and activations, LLMs also contain the self-attention layer, which involves two matrix multiplication operations between three activation tensors, i.e., query, key, and value. Given that the *BitMoD* PE only maintains one activation tensor in `FP16`, the other two activation tensors need to be low-precision integers. Fortunately, prior works have demonstrated that the key and value tensors are very amenable to quantization due to the softmax normalization inside self-attention, and can be safely quantized to `INT8` and even `INT4` with negligible accuracy loss [44], [52], [58]. Hence, *BitMoD* can accommodate the self-attention layer with the proposed bit-serial PE by quantizing the key and value tensors to low-precision integers.

C. BitMoD Accelerator

Fig. 6 shows the overall architecture of the *BitMoD* accelerator. The input and weight buffers are banked to provide adequate bandwidth for the access from PEs. The bit-serial term generator receives the weight data and decomposes them into bit-serial terms as discussed in Section IV-A. The main PE array contains 4×4 tiles connected in a systolic manner. Every PE tile has 8 rows $\times$ 8 columns and adopts an output-stationary dataflow. The bit-serial weight term is broadcast to the whole PE column, while the input is broadcast to the whole PE row. This allows *BitMoD* to exploit data reuse through both weight-sharing and input-sharing. Every PE column contains a local output buffer and an accumulator, which is used to accumulate the per-group partial sum from a PE to obtain the final per-channel output activation. Since processing a weight group takes many cycles, there is enough time to drain the whole PE column using only one shared accumulator to amortize the hardware cost.

V. Evaluation

A. Experimental Methodology

LLM Benchmarks. For evaluation, we choose six representative LLMs with diverse model sizes, including OPT-1.3B [57], Phi-2B [36], Yi-6B [1], Llama-2-7B, Llama-2-13B [34], and Llama-3-8B [35]. We obtain the pre-trained models from the HuggingFace repository, and implement the proposed *BitMoD* quantization framework in PyTorch. To evaluate the effects of quantization on the resulting model performance, we consider both discriminative and generative tasks. For discriminative tasks, we evaluate three benchmarks: HellaSwag [56], WinoGrande [41], and Piqa [7] under the zero-shot setting using LM-Evaluation-Harness [21]. For generative tasks, we choose Wikitext-2 [33] and C4 [18] datasets and evaluate the perplexity following the methodology in prior quantization works [4], [20], [30], [42].

Quantization Data Types. We compare the model accuracy of *BitMoD* with four baseline quantization data types:

- ANT [26], which adaptively uses different data types to quantize different tensors at the per-channel granularity.
- OliVe [25], that introduces an outlier-victim pair encoding mechanism, which sacrifices the normal value (i.e., victim) adjacent to the outlier to accommodate the important outlier value.
- Microscaling (MX) [40], which groups 32 low-precision FP weights with an extra 8-bit shared exponent.
- Per-group asymmetric integer quantization, which is commonly adopted in prior software quantization methods.

TABLE VII. Accuracy (higher is better) of discriminative tasks using different data types under per-group weight quantization.

Precision	Datatype	OPT-1.3B			Phi-2B			Yi-6B			Llama-2-7B			Llama-2-13B			Llama-3-8B			Mean △Acc
		Hella	Wino	Piqa	Hella	Wino	Piqa	Hella	Wino	Piqa	Hella	Wino	Piqa	Hella	Wino	Piqa	Hella	Wino	Piqa	
16-bit	FP16	53.72	59.43	72.41	73.74	75.77	79.22	74.96	70.72	78.78	75.98	69.06	79.11	79.39	72.38	80.5	79.18	72.85	80.74	0
4-bit	INT4-Asym	52.31	**59.35**	71.05	72.29	75.14	78.4	73.91	**70.51**	77.64	75.29	**68.74**	78.22	**78.76**	**72.45**	80.2	78.07	**73.24**	79.76	-0.71
	BitMoD	**53.03**	59.12	**71.49**	**72.51**	**77.58**	**79.48**	**73.98**	70.09	**78.35**	**75.43**	68.19	**78.45**	78.41	72.14	**80.42**	**78.49**	73.09	**79.98**	**-0.42**
3-bit	INT3-Asym	38.98	55.01	64.25	67.75	71.74	77.48	**71.3**	67.32	**76.71**	71.87	**66.46**	76.66	76.58	69.61	78.94	68.56	66.61	75.03	-4.84
	BitMoD	**49.16**	**58.09**	**68.88**	**70.16**	**75.22**	**78.18**	70.72	**67.72**	76.28	**72.68**	66.22	**77.53**	**76.79**	**72.37**	**79.22**	**73.56**	**70.32**	**77.91**	**-2.61**

To ensure a fair comparison with *BitMoD*, we only apply the data types of ANT, OliVe, and MX to LLM weight quantization while maintaining activations in `FP16`. Despite that the original ANT and OliVe frameworks only support per-channel quantization due to the absence of dedicated dequantization hardware, we have extended their algorithms for per-group quantization. This allows to compare the model accuracy purely based on the employed quantization data types. While we mainly focus on weight quantization in our evaluation, Section V-E demonstrates that *BitMoD* can be combined with SOTA activation quantization scheme, which offers the potential to reduce activation memory as well.

In addition to co-design works, we show that *BitMoD* can be seamlessly integrated into existing software-only quantization optimizations to further reduce the memory footprint or achieve better model performance. We combine *BitMoD* with three software quantization methods:

- SmoothQuant [52], which targets both weight and activation quantization in `INT8`. It addresses the quantization difficulty of large activation magnitude by partially migrating it to weights.
- AWQ [30], which employs activation-aware weight quantization to protect the salient weight channels corresponding to larger activation magnitudes.
- OmniQuant [42], which modulates the outlier weight values by optimizing the clipping threshold through block-wise fine-tuning.

To integrate *BitMoD* with these software-only methods, we replace their original weight quantizers that use integer data types with the extended `FP4` and `FP3` data types of *BitMoD*.

Accelerator Baselines. To evaluate the hardware performance and energy efficiency, we compare *BitMoD* with a baseline accelerator that supports `FP16` models and uses an `FP16` multiply-accumulate PE instead of the proposed bit-serial PE. We also compare *BitMoD* with ANT and OliVe, which design custom decoders to support multiple data types in a unified systolic array. We evaluate the performance in LLMs with a batch size of 1 and an input sequence length of 256, catering for edge use cases as in prior work [30].

Hardware Implementation. We implement the accelerator of *BitMoD* at RTL-level using SystemVerilog and verify the functionality of each component via RTL simulation. We use Synopsys Design Compiler to synthesize *BitMoD* in TSMC 28nm technology to report the area and power. For end-to-end performance evaluation, we implement a cycle-level simulator, where the accelerator timing and energy parameters are set based on the RTL synthesis results. The DRAM power is calculated based on the DDR4 model from DRAMSim3 [29]. All accelerators are evaluated under an iso-compute area constraint, and equipped with 512 KB activation buffer and 512 KB weight buffer, which are modelled with CACTI [6].

B. Accuracy Comparison of Different Data Types

Generative Tasks. Table VI details the perplexity of applying different PTQ data types at 4-bit and 3-bit weight precision. For 4-bit and 3-bit weight quantization, *BitMoD* achieves < 0.5 and < 3 perplexity loss on average compared to the `FP16` baseline models, respectively. Although ANT, OliVe, and MX are able to maintain acceptable perplexity at 4-bit precision, they experience significant degradation in perplexity when quantizing weights to 3 bits. OliVe, which is designed to handle outlier values under per-channel quantization, finds its advantages diminished since the impact of outliers can be significantly mitigated through per-group quantization as described in Section II-C. MX employs the basic `FP4` and `FP3` without exploring the potential of their redundant zero, leading to worse perplexity than `INT-Asym` that fully utilizes the available quantization levels while supporting asymmetry. On the contrary, *BitMoD* consistently outperforms asymmetric integer quantization at per-group granularity, and the benefits are more pronounced at 3-bit precision. This demonstrates that the combination of asymmetry and floating-point data types in *BitMoD* can significantly reduce the quantization error.

Discriminative Tasks. Table VII compares the model accuracy of discriminative tasks when employing *BitMoD* and the

TABLE VIII. Wikitext-2 and C4 perplexity (↓) when quantizing Llama weights using different data types. We use per-group weight quantization with a group size of 128.

Precision	Datatype	Llama-2-7B		Llama-2-13B		Llama-3-8B	
		Wiki	C4	Wiki	C4	Wiki	C4
4-bit	FP4	5.77	7.32	5.05	6.66	6.86	9.85
	FP4-ER	5.74	7.28	5.03	6.63	6.76	9.71
	FP4-EA	5.81	7.30	5.08	6.65	6.83	9.79
	BitMoD	**5.72**	**7.26**	**5.01**	**6.61**	**6.73**	**9.66**
3-bit	FP3	7.51	10.28	5.90	7.58	15.22	19.87
	FP3-ER	7.18	9.71	5.66	7.33	13.43	17.56
	FP3-EA	6.61	8.45	5.54	7.23	9.06	12.97
	BitMoD	**6.55**	**8.36**	**5.50**	**7.18**	**8.96**	**12.82**

TABLE IX. Wikitext-2 and C4 perplexity (↓) when using different special values for FP3.

Special Values	OPT-1.3B		Phi-2B		Llama-2-7B		Llama-3-8B	
	Wiki	C4	Wiki	C4	Wiki	C4	Wiki	C4
$\{\pm 5, \pm 6\}$	23.39	**20.12**	13.02	15.84	6.61	8.48	9.09	13.81
$\{\pm 3, \pm 5\}$	35.54	37.65	13.41	16.29	6.68	8.73	10.32	14.48
$\{\pm 3, \pm 6\}$	**22.67**	20.47	**12.91**	**15.69**	**6.55**	**8.36**	**8.96**	**12.82**

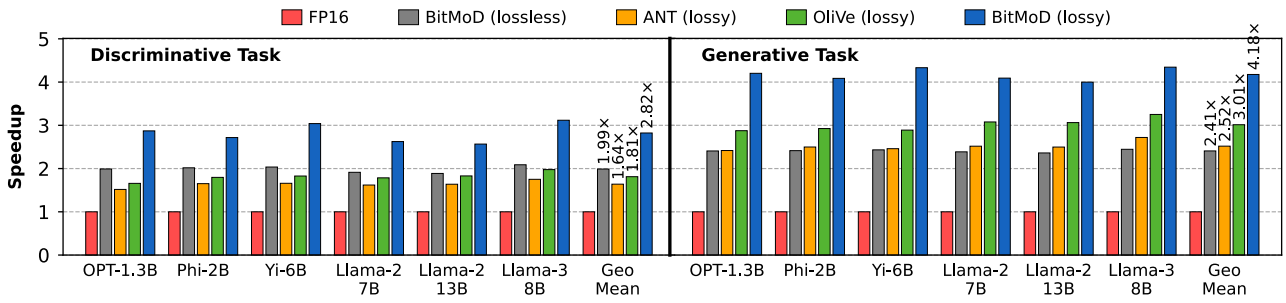

Fig. 7: Speedup of different accelerators (higher is better).

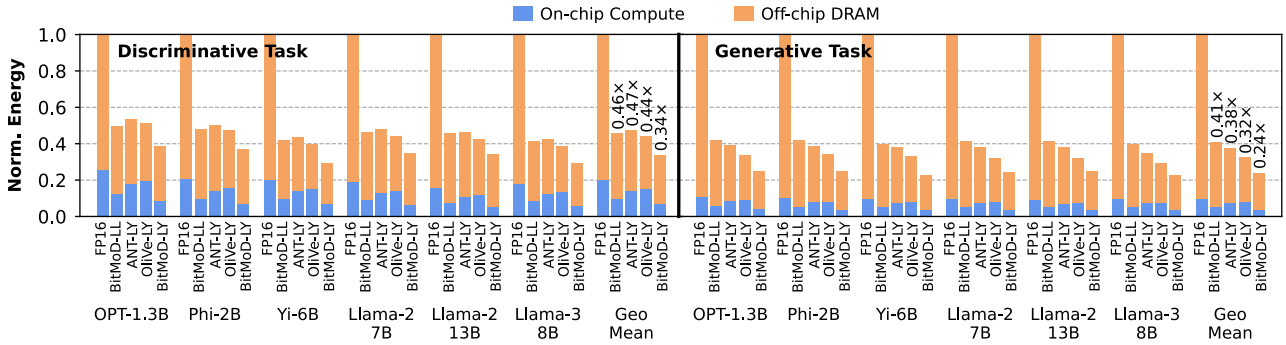

Fig. 8: Energy consumption breakdown of different accelerators (↓). "LL" and "LY" stand for 'lossless" and 'lossy", respectively.

baseline asymmetric integer quantization at per-group granularity. *BitMoD* achieves better or comparable accuracy than asymmetric integer quantization. At 4-bit precision, *BitMoD* has $< 0.5\%$ accuracy loss on average compared to the baseline `FP16` models. Moreover, on average, *BitMoD* achieves a big improvement of 2.2% in model accuracy compared to the asymmetric integer quantization that is widely used in SOTA software quantization methods.

***BitMoD* Data Type Ablation.** As discussed in Section III-A, *BitMoD* introduces new data types by adding extra resolution and asymmetry to `FP3` and `FP4`. We analyze the effects of these different data types on the perplexity of three studied Llama models. As shown in Table VIII, the *BitMoD* data types with both extra resolution and asymmetry achieve the best perplexity. Compared to the basic `FP4` data type, `FP4-ER` shows a greater improvement in perplexity than `FP4-EA`. This is because at 4-bit, the basic `FP4` still has enough quantization levels to quantize a weight group with asymmetric distribution, and adding extra resolution can better reduce the quantization error. On the contrary, for 3-bit precision, `FP3-EA` achieves much better perplexity than `FP3-ER`. Given fewer quantization levels at 3-bit, the extra asymmetry introduced by `FP3-EA` has a larger impact when accounting for weight groups with asymmetric distribution.

***BitMoD* Special Value Ablation.** The *BitMoD* PE can flexibly support different special values. We evaluate two other potential combinations of special values for `FP3`: $\{\pm 3, \pm 5\}$ and $\{\pm 5, \pm 6\}$. Table IX shows the resulting Wikitext and C4 perplexity. The adopted special values in *BitMoD*, i.e., $\{\pm 3, \pm 6\}$, achieve the lowest perplexity on average. The special value combination $\{\pm 5, \pm 6\}$ only introduces extra asymmetry to the basic `FP3`. However, many weight groups can exhibit symmetric distribution, which prefers a symmetric data type with the same absolute maximum and minimum values. On the other hand, the special values ± 5 have a higher quantization error than ± 6 as described in Section III-A, leading to worse perplexity when combined with ± 3.

C. Accelerator Performance

For accelerator evaluation, we consider two configurations for the *BitMoD* accelerator based on the resulting model accuracy: (1) *Lossless*, where the weight precision is `INT6` given its near-zero accuracy loss under per-group quantization. We compare this configuration with the baseline FP16 accelerator. (2) *Lossy*, where the weight can be quantized to 4-bit for discriminative tasks and 3-bit for generative tasks while maintaining good model performance. We compare this configuration with ANT and OliVe.

Tile Area and Power. Table X shows the PE tile area and power breakdown of *BitMoD* and the baseline FP16 accelerator at 1 GHz frequency. The unified bit-serial representation allows *BitMoD* to support different weight data types with low hardware cost. As a result, the *BitMoD* PE is 24% smaller

TABLE X. Area and power consumption per tile of the baseline FP16 accelerator vs. *BitMoD* at 1 GHz frequency.

	Number of PEs	Area (μm^2)			Power (mW)		
		PE Array	Encoder	Total	PE Array	Encoder	Total
Baseline	6×8	95,498	–	95,498	36.96	–	36.96
BitMoD	8×8	97,090	2,419	99,509	37.5	1.86	39.36

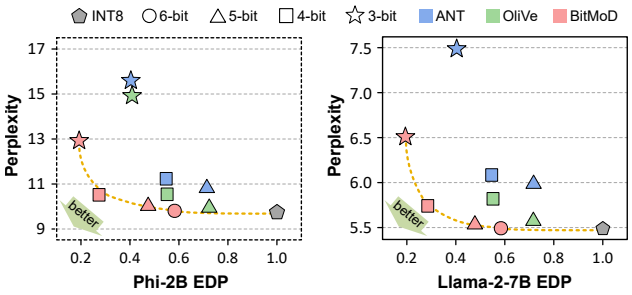

Fig. 9: Wikitext-2 perplexity-EDP pareto plot for Phi-2B and Llama-2-7B.

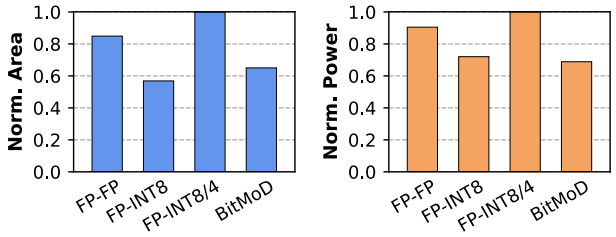

Fig. 10: Normalized area and power of *BitMoD* and different bit-parallel PEs.

than the baseline PE, which allows to fit more *BitMoD* PEs within the same compute area. Furthermore, the bit-serial term encoder has a tiny hardware overhead and only accounts for 2.5% of the PE array area.

Performance. Fig. 7 presents the hardware performance normalized to the baseline FP16 accelerator for discriminative and generative tasks. *BitMoD* achieves the best performance under both lossless and lossy quantization. The performance gain of *BitMoD* mainly comes from its careful algorithm-hardware co-design of different quantization data types. Since discriminative tasks are compute-bound and mainly involve matrix-matrix multiplications, the higher throughput of *BitMoD* PE leads to better performance than the baseline PE. In contrast, OliVe requires more complicated PEs with significant hardware overhead to accommodate the outliers, which have a much wider numerical range (e.g., $\{24, ..., 192\}$ at 4-bit precision). In comparison, the *BitMoD* data type has a small value range and can be efficiently processed with the proposed unified bit-serial representation. Regarding memory-bound generative tasks, *BitMoD* can quantize LLM weights to very low precision such as 3-bit, which offers significant memory saving while maintaining good perplexity. On the contrary, ANT and OliVe do not natively support per-group quantization due to the lack of dedicated dequantization hardware, and cannot maintain acceptable model quality under per-channel quantization using 3-bit weight precision. Therefore, they must adopt a higher weight precision to compensate for the significant degradation in perplexity. Overall, the lossless *BitMoD* achieves 1.99× and 2.41× speedup for discriminative and generative tasks, respectively compared to the baseline FP16 architecture. The lossy *BitMoD* achieves 1.72× / 1.56× and 1.66× / 1.39× speedup for discriminative and generative tasks, respectively compared to ANT / OliVe.

Energy Consumption. Fig. 8 presents the normalized energy breakdown of different accelerators, where the on-chip compute energy includes both buffer and core energy. The energy saving of *BitMoD* mainly comes from the reduced weight memory footprint and efficient bit-serial PE. Both ANT and OliVe require a higher weight precision than *BitMoD* to maintain acceptable model quality, leading to higher DRAM energy consumption. In addition, the baseline architecture uses `FP16` weights, which is an overkill for LLMs since the simple `INT6` data type can achieve comparable accuracy under per-group weight quantization. Overall, the lossless *BitMoD* achieves 2.31× better energy efficiency over the baseline architecture across different tasks. The lossy *BitMoD* has 1.48× and 1.31× better energy efficiency than ANT and OliVe, respectively.

Accuracy-Efficiency Trade-offs. The proposed *BitMoD* can offer good trade-offs between model accuracy and hardware efficiency. To demonstrate this, we analyze the relationship between energy-delay product (EDP) and model perplexity of Phi-2B and Llama-2-7B on Wikitext-2. We compare *BitMoD* with ANT and OliVe under different LLM weight precision. Fig. 9 shows the resulting perplexity-EDP relationship for the studied two LLMs and three accelerators. Note that while the 5-bit precision is not presented explicitly, *BitMoD* can be easily extended to perform bit-serial `INT5` computation using its Booth encoder. Similarly, the custom data types introduced by ANT and OliVe can be extended to 5-bit precision based on their data type definition. As indicated in Fig. 9, the lower left region indicates a *better* trade-off between perplexity and EDP. Although ANT and OliVe propose different algorithm-hardware co-design approaches for LLM acceleration, they only leverage the per-channel quantization granularity that fails to preserve the model quality at very low precision, and they lack a unified architecture to efficiently support different data types and precision. In contrast, *BitMoD* exploits new data types tailored for per-group quantization and adopts an efficient bit-serial computing paradigm to support various data types. Hence, *BitMoD* can always sit on the Pareto frontier.

D. Comparison to Mixed-Precision Bit-Parallel Architecture

FIGNA [27] proposes a family of bit-parallel PEs for arithmetic between low-precision integer weight and floating-point activation. However, every FIGNA PE is designed separately and only supports one weight precision, which fails to offer a trade-off between model accuracy and hardware efficiency. We explore the possibility of FIGNA to support mixed-precision integer weights. We consider a baseline FIGNA-like PE that performs multiply-accumulate between `FP16` activation and `INT8` weight. We extend the baseline PE to support either one `FP16-INT8` operation or two `FP16-INT4` operations, which multiply the same `FP16` activation with two `INT4` weights. Fig. 10 compares the normalized area and power of different PEs. Although the `FP-INT8` PE has the smallest area, adding support for mixed weight precision incurs significant hardware overhead and leads to even higher area and power consumption than the conventional `FP-FP` PE. This is because a bit-parallel PE computing two `FP16-INT4` operations will

TABLE XI. Wikitext-2 and C4 perplexity (↓) of different quantization strategies. We use per-group weight quantization with a group size of 128. For every table column, we highlight the best two perplexity results in bold.

Bits	Method	Llama-2-7B		Llama-2-13B		Llama-3-8B		Mean
		Wiki	C4	Wiki	C4	Wiki	C4	ΔPPL
16-bit	FP16	5.47	6.97	4.88	6.47	6.13	8.88	0
4-bit	QuaRot	5.60	7.48	5.00	6.88	6.54	10.18	0.48
	GPTQ	5.63	7.13	4.99	6.56	6.53	9.38	0.24
	AWQ	5.60	7.12	4.97	6.56	6.54	9.39	0.23
	OmniQ	5.59	7.12	4.96	6.56	6.57	9.50	0.25
	BitMoD + AWQ	**5.59**	**7.09**	**4.96**	**6.55**	**6.50**	**9.33**	**0.20**
	BitMoD + OmniQ	**5.57**	**7.07**	**4.95**	**6.55**	**6.45**	**9.30**	**0.18**
3-bit	QuaRot	6.09	8.44	5.37	7.52	**7.64**	12.49	1.88
	GPTQ	6.29	7.89	5.42	7.00	9.58	11.66	1.51
	AWQ	6.24	7.81	5.32	6.95	8.22	11.56	1.22
	OmniQ	**6.05**	7.76	5.28	6.99	8.33	12.04	1.28
	BitMoD + AWQ	6.07	**7.64**	**5.27**	**6.88**	7.81	**11.07**	**0.98**
	BitMoD + OmniQ	**5.89**	**7.59**	**5.21**	**6.85**	**7.57**	**11.05**	**0.89**

produce two separate outputs, doubling the cost of a floating-point accumulator and output register. In contrast, the bit-serial *BitMoD* PE trades-off between weight precision and latency, which requires only one accumulator and output register for any weight precision. Consequently, *BitMoD* offers the highest flexibility to support variable weight precision with better efficiency compared to the decomposable bit-parallel PE.

E. Combining BitMoD with Other Quantization Schemes

BitMoD can be seamlessly integrated with existing software-only quantization methods by replacing their original weight quantizers that use integer data types with the extended `FP4` and `FP3` data types of *BitMoD*. We demonstrate such feasibility on AWQ [30], OmniQuant [42], and SmoothQuant [52].

Orthogonal to Quantization Optimization. The original AWQ and OmniQuant adopt `INT-Asym` weight quantization with several algorithmic optimizations such as weight clipping and scaling factor search, leading to SOTA model performance under 4-bit and 3-bit weight precision. We evaluate the model performance when applying AWQ and OmniQuant optimizations on top of the *BitMoD* data type. We also compare with SOTA software-only LLM quantization methods, including GPTQ [20] and QuaRot [4] under weight-only quantization. Table XI shows the Wikitext and C4 perplexity of the studied Llama models using different quantization strategies. Combining *BitMoD* with AWQ and OmniQuant significantly outperforms other approaches. For example, applying the *BitMoD* data type reduces the average perplexity loss of OmniQuant by 28% and 31% at 4-bit and 3-bit precision, respectively. Overall, *BitMoD* combined with AWQ and OmniQuant achieves an average perplexity loss of < 1 for both 4-bit and 3-bit weight precision, pushing the limit of LLM weight quantization to a new state-of-the-art. It's important to note that using AWQ and OmniQuant does not inhibit the functionality of the *BitMoD* accelerator—their optimization merely adjusts the per-group scaling factor, which is supported by the bit-serial dequantization unit of *BitMoD*.

TABLE XII. Wikitext-2 perplexity (↓) when activation maintains in FP16 or is quantized to INT8 with SmoothQuant (SQ8). For weights, we use per-group quantization with a group size of 128.

Weight Precision	Weight Datatype	Llama-2-7B		Llama-2-13B		Llama-3-8B	
		FP16	SQ8	FP16	SQ8	FP16	SQ8
8-bit	INT8	5.47	5.52	4.95	4.93	6.13	6.26
4-bit	INT4-Asym	5.77	5.83	5.01	5.09	6.84	7.05
	BitMoD	5.72	5.76	5.01	5.07	6.73	6.87
3-bit	INT3-Asym	7.08	7.58	5.64	5.99	13.26	25.78
	BitMoD	6.55	6.85	5.5	5.82	9.09	10.57

Orthogonal to Activation Quantization. SmoothQuant can quantize LLM activation to `INT8` with low accuracy loss. We conduct weight PTQ using *BitMoD* and `INT-Asym` data types on the pre-calibrated Llama models from SmoothQuant that use `INT8` activation. Table XII shows the WikiText-2 perplexity under `FP16` and `INT8` activation using different quantized weight precision and data types. The perplexity improvement of *BitMoD* over `INT-Asym` remains after quantizing activation to `INT8` using SmoothQuant, and the improvement is particularly pronounced at lower precision (i.e., 3-bit). For instance, on Llama-3-8B, *BitMoD* improves the perplexity by 15.21 compared to `INT3-Asym` after applying SmoothQuant. Notably, on Llama-2-7B, the perplexity of *BitMoD* weight with `INT8` activation is even better than `INT-Asym` weight with `FP16` activation, which demonstrates the potential of *BitMoD* for further reducing the LLM memory footprint under a target model quality.

VI. Related Work

DNN Accelerators. There is an abundance of prior work on DNN accelerators [3], [5], [12], [23]–[28], [39], [40], [43], [45], [48], [50], [54], [55], [59]. These accelerators propose specialized processing elements and data flow to match the computational characteristics and memory access pattern of DNNs. Some accelerators exploit value sparsity to accelerate small-scale DNNs with the help of retraining [23], [24], [48], [50], [59]. Other works target low-precision DNN acceleration based on model quantization [25]–[27], [39], [40], [54], [55]. Among them, [25], [26], [40] introduce custom data types to better fit the value distribution of DNNs. Another line of works relies on bit-serial computing to scale the performance with lower operand precision, and leverages bit-level sparsity to skip ineffectual bit operations [3], [5], [12], [28], [43], [45]. The proposed *BitMoD* combines the benefits of quantization and bit-serial computing to efficiently trade-off between weight precision and hardware efficiency.

LLM Quantization. Numerous algorithmic studies have proposed quantization solutions to reduce the memory footprint of LLMs [4], [11], [15], [19], [20], [30], [31], [42], [49], [52]. Most of these works rely on asymmetric integer quantization for LLM weights while applying other techniques to optimize the quantization parameters. The proposed *BitMoD* data type is orthogonal to many of these works and can be synergistically combined with different quantization optimizations.

VII. Conclusion

In this paper, we introduce *BitMoD*, an algorithm-hardware co-design scheme for efficient LLM acceleration. On the algorithm side, *BitMoD* designs new data types that are tailored for per-group LLM weight quantization at very low precision. By intelligently repurposing the redundant zero value to an additional number, *BitMoD* extends the resolution or range of 3-bit-and 4-bit floating-point data types. Moreover, the *BitMoD* quantization framework can be seamlessly integrated with existing software-only quantization methods to further improve the model performance. On the hardware side, *BitMoD* proposes a unified bit-serial representation for diverse low-precision data types and an efficient bit-serial PE to process quantized weight and FP16 activation. Our evaluation demonstrates that *BitMoD* significantly outperforms existing LLM quantization methods, pushing the limit of LLM weight quantization to a new state-of-the-art. Compared to prior accelerators ANT / OliVe, *BitMoD* achieves $1.69\times$ / $1.48\times$ speedup and $1.48\times$ / $1.31\times$ better energy efficiency, while being able to support diverse weight precision to offer a good trade-off between model accuracy and hardware efficiency.

Acknowledgment

This project is supported in part by Intel Corporation, the National Science Foundation under Grant No. 2339084, and the Engineering and Physical Sciences Research Council (EPSRC) under Grant No. EP/S030069/1. We would like to thank Jordan Dotzel, Hongxiang Fan, Stylianos I. Venieris, Alexandros Kouris, Mahesh Iyer, Grace Zgheib, Sergey Gribok, and the anonymous reviewers for their constructive feedback.

References

[1] 01-AI, "01-ai/yi-6b." [Online]. Available: https://huggingface.co/01-ai/Yi-6B

[2] Abdelfattah Lab, "BitMoD Artifacts," Nov. 2024. [Online]. Available: https://doi.org/10.5281/zenodo.14252531

[3] J. Albericio, A. Delmas, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-pragmatic deep neural network computing," *IEEE/ACM 50th Annual International Symposium on Microarchitecture (MICRO)*, 2017.

[4] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, "QuaRot: Outlier-free 4-bit inference in rotated llms," *arXiv preprint arXiv:2404.00456*, 2024.

[5] O. M. Awad, M. Mahmoud, I. E. Vivancos, A. H. Zadeh, C. Bannon, A. Jayarajan, G. Pekhimenko, and A. Moshovos, "FPRaker: A processing element for accelerating neural network training," *IEEE/ACM 54th Annual International Symposium on Microarchitecture (MICRO)*, 2020.

[6] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "CACTI 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, June 2017.

[7] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, "PIQA: Reasoning about physical commonsense in natural language," *arXiv preprint arXiv:1911.11641*, 2019.

[8] A. D. Booth, "A signed binary multiplication technique," *Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, pp. 236–240, 1951.

[9] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, 2020, pp. 1877–1901.

[10] J. Chee, Y. Cai, V. Kuleshov, and C. D. Sa, "QuIP: 2-bit quantization of large language models with guarantees," *arXiv preprint arXiv:2307.13304*, 2023.

[11] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K.-C. Zhang, Y. Qiao, and P. Luo, "EfficientQAT: Efficient quantization-aware training for large language models," *arXiv preprint arXiv:2407.11062*, 2024.

[12] Y. Chen, J. Meng, J.-S. Seo, and M. S. Abdelfattah, "BBS: Bi-directional bit-level sparsity for deep learning acceleration," *57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.

[13] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? try ARC, the ai2 reasoning challenge," *arXiv preprint arXiv:1803.05457*, 2018.

[14] S. Dai, R. Venkatesan, H. Ren, B. Zimmer, W. J. Dally, and B. Khailany, "VS-Quant: Per-vector scaled quantization for accurate low-precision neural network inference," in *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

[15] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," *arXiv preprint arXiv:2208.07339*, 2022.

[16] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," *arXiv preprint arXiv:2110.02861*, 2022.

[17] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized llms," *arXiv:2305.14314*, 2023.

[18] J. Dodge, A. Marasovic, G. Ilharco, D. Groeneveld, M. Mitchell, and M. Gardner, "Documenting large webtext corpora: A case study on the colossal clean crawled corpus," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[19] J. Dotzel, Y. Chen, B. Kotb, S. Prasad, G. Wu, S. Li, M. S. Abdelfattah, and Z. Zhang, "Learning from students: Applying t-distributions to explore accurate and efficient formats for llms," *arXiv preprint arXiv:2405.03103*, 2024.

[20] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training compression for generative pretrained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[21] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, "A framework for few-shot language model evaluation," 12 2023. [Online]. Available: https://zenodo.org/records/10256836

[22] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "Ai and memory wall," *IEEE Micro*, 2024.

[23] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.

[24] A. Gondimalla, M. Thottethodi, and T. N. Vijaykumar, "Eureka: Efficient tensor cores for one-sided unstructured sparsity in dnn inference," *56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.

[25] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y.-B. Liu, M. Guo, and Y. Zhu, "OliVe: Accelerating large language models via hardware-friendly outlier-victim pair quantization," *ACM/IEEE 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023.

[26] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y.-B. Liu, M. Guo, and Y. Zhu, "ANT: Exploiting adaptive numerical data type for low-bit deep neural network quantization," *IEEE/ACM 55th Annual International Symposium on Microarchitecture (MICRO)*, 2022.

[27] J. Jang, Y. Kim, J. Lee, and J.-J. Kim, "FIGNA: Integer unit-based accelerator design for fp-int gemm preserving numerical accuracy," *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.

[28] P. Judd, J. Albericio, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.

[29] S.-J. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "DRAMsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, pp. 106–109, 2020.

[30] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for llm compression and acceleration," in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[31] Z. Liu, B. Oğuz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, "LLM-QAT: Data-free quan-

tization aware training for large language models," *arXiv preprint arXiv:2305.17888*, 2023.
[32] J. Meng, Y. Liao, A. Anupreetham, A. Hasssan, S. Yu, H.-s. Suh, X. Hu, and J.-s. Seo, "Torch2Chip: An end-to-end customizable deep neural network compression and deployment toolkit for prototype hardware accelerator design," in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
[33] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," *arXiv preprint arXiv:1609.07843*, 2016.
[34] Meta, "Meta llama." [Online]. Available: https://github.com/meta-llama/llama
[35] Meta, "Meta llama 3." [Online]. Available: https://github.com/meta-llama/llama3
[36] Microsoft, "microsoft/phi-2." [Online]. Available: https://huggingface.co/microsoft/phi-2
[37] NVIDIA, "Jetson TX2 Module." [Online]. Available: https://developer.nvidia.com/embedded/jetson-tx2
[38] Open Compute Project, "OCP Microscaling Formats (MX) Specification." [Online]. Available: https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf
[39] E. Park, D. Kim, and S. Yoo, "Energy-efficient neural network accelerator based on outlier-aware low-precision computation," *ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018.
[40] B. D. Rouhani, R. Zhao, V. Elango, R. Shafipour, M. Hall, M. Mesmakhosroshahi, A. More, L. Melnick, M. Golub, G. Varatkar, L. Shao, G. Kolhe, D. Melts, J. Klar, R. L'Heureux, M. Perry, D. Burger, E. S. Chung, Z. Deng, S. Naghshineh, J. Park, and M. Naumov, "With shared microexponents, a little shifting goes a long way," *ACM/IEEE 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023.
[41] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, "WinoGrande: An adversarial winograd schema challenge at scale," *arXiv preprint arXiv:1907.10641*, 2019.
[42] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. J. Qiao, and P. Luo, "OmniQuant: Omnidirectionally calibrated quantization for large language models," *arXiv preprint arXiv:2308.13137*, 2024.
[43] S. Sharify, A. D. Lascorz, M. Mahmoud, M. Nikolic, K. Siu, D. M. Stuart, Z. Poulos, and A. Moshovos, "Laconic deep learning inference acceleration," *ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, 2019.
[44] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, Z. Xie, B. Chen, C. W. Barrett, J. Gonzalez, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning (ICML)*, 2023.
[45] M. Shi, V. Jain, A. Joseph, M. Meijer, and M. Verhelst, "BitWave: Exploiting column-based bit-level sparsity for deep learning acceleration," *Proceedings of the 30th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.
[46] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2013.
[47] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
[48] Y. Wang, C. Zhang, Z. Xie, C. Guo, Y. Liu, and J. Leng, "Dual-side sparse tensor core," *ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.
[49] X. Wu, H. Xia, S. Youn, Z. Zheng, S. Chen, A. Bakhtiari, M. Wyatt, R. Y. Aminabadi, Y. He, O. Ruwase, L. Song, and Z. Yao, "ZeroQuant(4+2): Redefining llms quantization with a new fp6-centric strategy for diverse generative tasks," *arXiv preprint arXiv:2312.08583*, 2023.
[50] Y. N. Wu, P.-A. Tsai, S. Muralidharan, A. Parashar, V. Sze, and J. S. Emer, "HighLight: Efficient and flexible dnn acceleration with hierarchical structured sparsity," *56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.
[51] H. Xia, Z. Zheng, X. Wu, S. Chen, Z. Yao, S. Youn, A. Bakhtiari, M. Wyatt, D. Zhuang, Z. Zhou, O. Ruwase, Y. He, and S. L. Song, "Quant-llm: Accelerating the serving of large language models via fp6-centric algorithm-system co-design on modern gpus," in *USENIX Annual Technical Conference (ATC)*, 2024.
[52] G. Xiao, J. Lin, M. Seznec, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," *arXiv preprint arXiv:2211.10438*, 2022.
[53] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers," *arXiv preprint arXiv:2206.01861*, 2022.
[54] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "GOBO: Quantizing attention-based nlp models for low latency and energy efficient inference," *IEEE/ACM 53rd Annual International Symposium on Microarchitecture (MICRO)*, 2020.
[55] A. H. Zadeh, M. Mahmoud, A. Abdelhadi, and A. Moshovos, "Mokey: enabling narrow fixed-point inference for out-of-the-box floating-point transformer models," *ACM/IEEE 49th Annual International Symposium on Computer Architecture (ISCA)*, 2022.
[56] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a machine really finish your sentence?" in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
[57] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. T. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.
[58] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit quantization for efficient and accurate llm serving," in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
[59] X. Zhou, Z. Du, Q. Guo, S. Liu, C. Liu, C. Wang, X. Zhou, L. Li, T. Chen, and Y. Chen, "Cambricon-S: Addressing irregularity in sparse neural networks through a cooperative software/hardware approach," *51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018.

APPENDIX

A. Abstract

This artifact appendix describes how to access and evaluate the BitMoD artifact [2] to reproduce the experiments as performed in Section V.

B. Artifact Check-List (Meta-Information)

- **Program:** Python, Shell script
- **Runtime Environment:** Ubuntu 20.04
- **Hardware:** NVIDIA GPU with $\geq$40 GB of VRAM (e.g., A6000).
- **Model (LLM):** OPT-1.3B, Phi-2B, Yi-6B, Llama-2-7B, Llama-2-13B, Llama-3-8B.
- **Dataset:** Wikitext, C4.
- **Experiments:** LLM perplexity evaluation in Section V-B and Section V-E. Accelerator performance evaluation in Section V-C.
- **Disk space required (approximately):** 512 GB.
- **How much time is needed to complete experiments (approximately)?:** 50 hours.
- **Publicly available?:** Yes.
- **Archived:** https://doi.org/10.5281/zenodo.14252531

C. Description

1) How to Access: We maintain a publicly-available repository [2] where we have open-sourced all of our artifacts.

2) Hardware Dependencies: One NVIDIA GPU with at least 40 GB of VRAM (e.g., A6000), in addition to a normal desktop computer with at least 512 GB of free disk space.

3) Software Dependencies: All experiments require Conda for managing virtual Python environments. The quantization experiments require CUDA. Other requirements are automatically installed by scripts in the following sections. When running experiments, please use a tmux session to make sure long-running jobs are not killed unexpectedly.

D. Installation

For artifact evaluation, begin by downloading the top-level repository from Zenodo:

```
$ wget -O BitMoD.zip https://zenodo.org/records/14252531/files/BitMoD-HPCA-25.zip
$ unzip BitMoD.zip
```

The artifact is inside the unzipped **BitMoD-HPCA-25** repository, which contains five sub-folders, each targeting one set of experiments:

1) **bitmod-quant**, which runs the baseline weight-only quantization with different data types. This can reproduce the results in Table VI and Table VIII.
2) **bitmod-sim**, contains a custom simulator that calculates the latency and energy of the *BitMoD* accelerator. This can reproduce the results in Fig. 7 and Fig. 8.
3) **AWQ-BitMoD**, which runs AWQ [30] with integer and *BitMoD* data types. This can reproduce the AWQ results in Table XI.
4) **OmniQuant-BitMoD**, which runs OmniQuant [42] with integer and *BitMoD* data types. This can reproduce the OmniQuant results in Table XI.
5) **SmoothQuant-BitMoD**, which runs SmoothQuant [52] with integer and *BitMoD* data types for weight quantization. This can reproduce the results in Table XII.

Please go to every sub-folder and refer to the corresponding '`README.md`' for detailed setup instructions. Note that AWQ, OmniQuant, and SmoothQuant will require different conda environments. Hence, please change back to the base environment after installing a conda environment before creating the next.

For example, the AWQ environment can be created by running:

```
$ cd AWQ-BitMoD
$ conda create -n awq-bitmod python=3.10 -y
$ conda activate awq-bitmod
# Follow 'README.md' inside AWQ-BitMoD folder to set up other dependencies.
$ conda deactivate # change back to the base environment
```

The OmniQuant and SmoothQuant environments can be created in a similar way by repeating the above step and following their '`README.md`' to set up other dependencies.

E. Experiment workflow

Once the environment is set up, we will conduct five sets of experiments, each corresponding to one of the five sub-folders within the **BitMoD-HPCA-25** repository.

1) BitMoD Weight-only Quantization: Run the basic LLM weight-only quantization experiments to reproduce the results in Table VI and Table VIII.

```
$ cd bitmod_quant
$ conda activate awq-bitmod
```

In '`run_exp.sh`', modify the '`export`' command by specifying the HuggingFace home directory, '`HF_HOME`', on your computer. By default, this can be set to your home directory.

```
$ export HF_HOME="your/HF_HOME/directory"
```

Then run the following:

```
$ bash run_exp.sh
```

The perplexity result will be saved in the folder called '`results_quant`'.

2) BitMoD Hardware Simulation: Before running the simulator, go to '`bitmod_sim`' of the repository:

```
$ cd bitmod_sim
$ conda activate awq-bitmod
```

In 'run_shape_profile.sh', modify the 'export' command by specifying the HuggingFace home directory, 'HF_HOME', on your computer:

```
$ export HF_HOME="your/HF_HOME/directory"
```

Then generate the model shape information that can be passed to the accelerator simulator:

```
$ bash run_shape_profile.sh
```

Next, run different simulators for the baseline FP16 accelerator, ANT, OliVe, and *BitMoD*:

```
$ python test_baseline.py --is_generation
$ python test_ant.py --is_generation
$ python test_olive.py --is_generation
$ python test_bitmod.py --is_generation --is_lossless
```

The flag *--is_generation* is optional. When enabled / disabled, it will evaluate the hardware performance of generative / discriminative tasks. The flag *--is_lossless* is optional for *BitMoD*. When enabled / disabled, it will evaluate the hardware performance of lossless / lossy *BitMoD* quantization.

Finally, to generate Fig. 7 and Fig. 8 of the paper, go to 'bitmod_sim/plot' directory and run the Jupyter notebooks inside. Note that the cycle and energy numbers are the same as those output by the simulators.

3) AWQ: Go to the 'AWQ-BitMoD' directory:

```
$ cd AWQ-BitMoD
$ conda activate awq-bitmod
```

In 'run_awq.sh' and 'run_eval_ppl.sh', modify the first 'export' command by specifying the HuggingFace home directory, 'HF_HOME', on your computer:

```
$ export HF_HOME="your/HF_HOME/directory"
```

Then, run the following two commands separately:

```
$ bash run_awq.sh # will take several hours
$ bash run_eval_ppl.sh
```

The perplexity results will be saved in the folder called 'results'. You can compare these with the AWQ results in Table XI.

4) OmniQuant: Go to the 'OmniQuant-BitMoD' directory:

```
$ cd OmniQuant-BitMoD
$ conda activate omniquant-bitmod
```

The comprehensive scripts to reproduce the Table XI OmniQuant results are available in the 'scripts' directory. Before running any command in the scripts, execute the following 'export' command and specify the HuggingFace home directory, 'HF_HOME', on your computer:

```
$ export HF_HOME="your/HF_HOME/directory"
```

In every shell script, you need to change the parameter of *--model* flag to the LLM path in your computer. By default, this can be set to the LLM directory from the official HuggingFace website. For example, inside the script 'llama-2-13b-int.sh', the Llama-2-7B model path can be specified with:

```
--model meta-llama/Llama-2-7b-hf
```

After changing the *--model* flag to the correct model path, copy and execute every script's python command under the 'OmniQuant-BitMoD' directory. You may check the perplexity results at the end of the log file specified by the *--output_dir* flag in every command, and compare those with the OmniQuant results in Table XI.

5) SmoothQuant: Go to the 'SmoothQuant-BitMoD' directory:

```
$ cd SmoothQuant-BitMoD
$ conda activate smoothquant-bitmod
```

In 'run_experiments.sh', modify the 'export' command by specifying the HuggingFace home directory, 'HF_HOME', on your computer:

```
$ export HF_HOME="your/HF_HOME/directory"
```

Then, run the following command:

```
$ bash run_experiments.sh
```

The perplexity results will be saved in the folder called 'results_mod'. You can compare these results with the SmoothQuant results in Table XII.

F. Evaluation and expected results

There are five result folders after running the above experiments:

1) bitmod-quant/results_quant, contains the perplexity results in Table VI and Table VIII.
2) bitmod-sim/plot, contains two Jupyter notebooks to reproduce Fig. 7 and Fig. 8, respectively.
3) AWQ-BitMoD/results, contains the AWQ results in Table XI.
4) OmniQuant-BitMoD/log, contains the OmniQuant results in Table XI.
5) SmoothQuant-BitMoD/results_mod, contains the SmoothQuant results in Table XII.

G. Methodology

Submission, reviewing and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-and-badging-current
- https://cTuning.org/ae